

软件工程复习指南

软件工程题型及分值介绍（共 30 分）

单项选择题（每小题 1 分，共 5 分）

判断题（每小题 1 分，共 5 分）

简答题（每小题 4 分，共 12 分）

建模题或解答题（共 1 题，一般 3 个子问题共 8 分（3、2、3））

重点标识：★ 星数越多越重点

此文档只针对国考复习

更新日期	更新内容	说明
20220225	初次编写	
20230311	补充、修订	

目录

软件工程复习指南	1
软件工程题型及分值介绍（共 30 分）	1
一、 软件工程概念及基本要素	3
二、 软件需求与软件需求规约	3
三、 软件过程	3
四、 结构化方法	7
五、 UML 面向对象建模	9
六、 软件测试	15
七、 项目管理	19
八、 软件质量	20

一、 软件工程概念及基本要素

考点：不考大题，几乎只有少量选择和判断，没空可以不看，有空大致翻翻

1、 软件的概念 ★

软件是指程序、数据、文档或者是指程序、文档

其中文档有：开发文档、管理文档、用户文档

2、 软件的正确性 ★★★

软件的正确性是指软件达到预期功能的程度。

3、 软件工程的基本活动

① 需求(规格)、②设计(解决方案)、③验证(验证规格是否符合)、④软件进化(维护)

二、 软件需求与软件需求规约

考点：软件需求规格说明书

1、 软件需求规格说明书 ★★★★★

软件需求规格说明书是由开发人员经需求分析后形成的软件文档，是对需求分析工作的全面总结，是软件需求分析的最终结果。

软件需求规格说明书包括：概述、功能性需求、非功能性需求、约束等几大块。

概述：描述系统的上下文、关键性功能场景、角色以及能够使用的功能，即用例；

功能性需求：描述用例、报表、接口三大类；

非功能性需求：性能、安全等，视情况而定；

约束：同非功能性需求一样，需要才补充，有技术型、软硬件、使用场景、法律法规等

2、 软件可行性研究 ★

可行性研究的目的是用最小的代价在尽可能短的时间内确定问题是否能够解决。

① 经济可行性：进行成本/效益分析，系统的经济效益能否超过开发成本；

② 技术可行性：进行技术风险评价，使用现有的技术能否实现这个系统；

③ 法律可行性：确定系统开发可能导致的任何侵权、妨碍和责任；

④ 操作可行性：系统的操作方式在该客户组织内是否行得通。

3、 软件开发需求活动 ★★

软件开发需求活动包括：需求获取、需求定义、需求分析、需求验证；主要目的是定义需求并建立系统模型

三、 软件过程

软件开发活动各个阶段：需求阶段-定义需求并建立系统模型，编制需求规约；设计阶段-给出软件解决方案；总体设计-给出软件体系结构，设计各模块；详细设计-给出系统模块结构和定义模块的算法；实现-软件编码；确认-软件测试；支持-完善性维护，纠错性维护。

软件过程

在软件生存周期中所有实施的一系列活动的集合，且每个活动可由一些任务完成。

一套流程、规章制度；在软件生存周期中所有活动的集合。

软件过程能力

按照规则制度标准达到预期目标的能力

软件生存周期

考点：软件生存周期的基本过程（5个）

1、软件生产周期概念

软件产品从构思开始到不再使用的时间周期。软件生存周期典型的阶段包括：需求阶段、设计阶段、实现阶段、测试阶段、安装与验收阶段、操作与维护阶段，有时还包括退役阶段

2、软件生存周期过程 ★★★★★

- ① **基本过程（5个）**：是软件开发人员从事的活动。包括获取过程、供应过程、开发过程、运作过程、维护过程
- ② **支持过程（9个）**：软件需求方和开发方各类支持人员从事的活动。包括文档编制过程、配置管理过程、质量保证过程、验证过程、确认过程、联合评审过程、审核过程、问题解决过程、易用性过程
- ③ **组织过程（7个）**：管理人员从事的活动。包括管理过程、基础设施过程、改进过程、人力资源过程、资产管理过程、重用大纲管理过程、领域工程过程

软件生命周期模型

考点：软件生命周期模型：瀑布模型、演化模型、增量模型、喷泉模型，以选择、判断为主

1、软件生命周期模型概念

软件生命周期模型也叫软件开发模型，是软件开发全部过程、活动和任务的结构框架，能清晰、直观表达软件开发全过程，明确规定了要完成的主要活动和任务，用来做为软件开发的基础。

2、软件开发模型 必考★★★★★★★★★

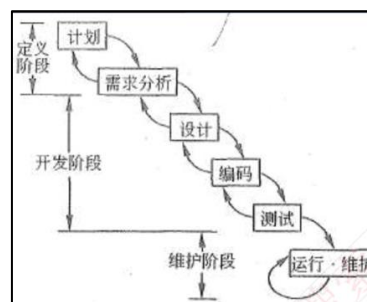
① 瀑布模型

适用于**需求明确**，需求变更小，**安全性要求高**的项目；

严格遵循软件生命周期每个阶段的固定顺序，前一阶段完成后才进入下一阶段。

步骤：需求分析（验证）→ 规格说明（验证）→ 设计（验证）→ 编码（验证）→ 综合测试→维护

特点：阶段间具有顺序性和依赖性；质量保证的观点；推迟实现的观点（编码方面，考虑返工、成本、效益）

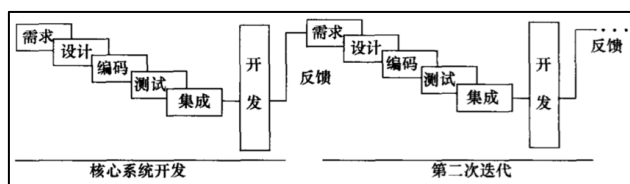


② 演化模型

适用于**需求不明确**的项目，在用户提出的核心需求的基础上先开发一个版本，根据

用户反馈后，再实施**迭代开发**，一定程度上减少软件开发活动的盲目性；在应用演化模型时，可以使用**瀑布模型**来管理每个演化的增量；产出结果给测试。

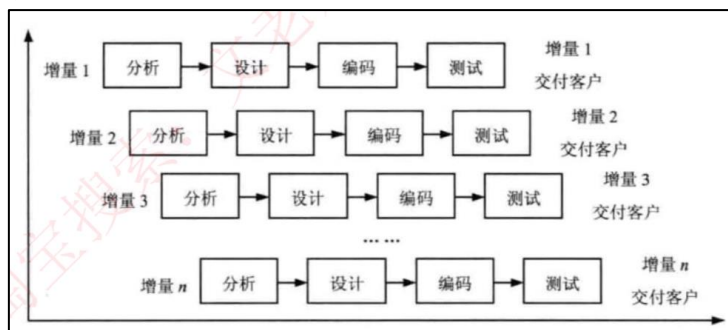
特点：分阶段、实验性、渐增式演化型



③ 增量模型

需求明确，产品分解成若干个构件**逐步交付**，可以得到初步可使用的版本；多条线并行；风险低；**产出结果交付给客户**。

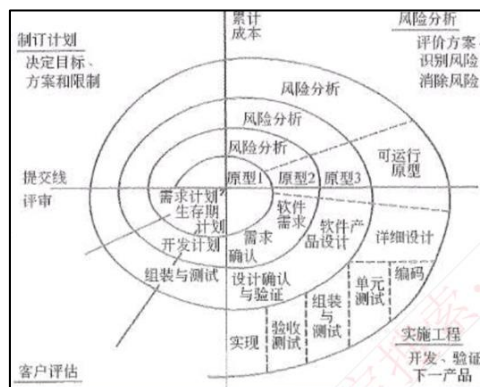
首先开发核心模块功能，而后与用户确认（用户参与进来），再次开发核心模块，最终完成项目，优先级最高的服务先交付，不利于模块划分，**每次增量版本都可以独立可操作的产品**。



④ 螺旋模型

适用于**需求不明确、项目大、风险高**的项目；

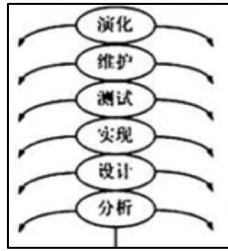
瀑布模型+演化模型+风险评估



⑤ 喷泉模型

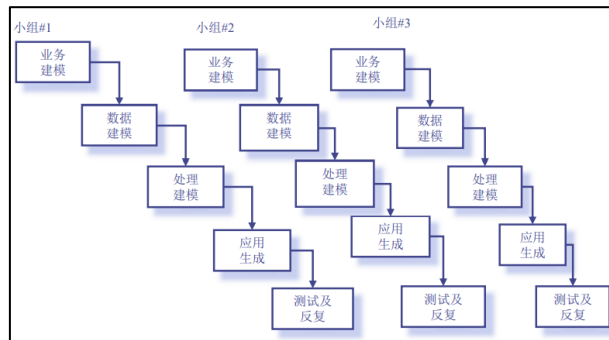
适用于**面向对象**的开发方法；具有迭代性和无间隙性。

特点：各个阶段没有明显的界限，可以同步进行，提高开发效率。由于各个开发阶段是重叠的，因此需要大量开发人员，不利于项目管理。



⑥ 快速应用开发模型（RAD）

适用于极短的开发周期，迅速得到结果；多条可以并行进行。



演化模型与增量模型的区别：相同点：非整体开发，以渐增式开发系统；不同点：增量模型：产出结果给客户使用；演化模型：多次迭代，产出结果给测试

软件过程模型

ISO9000

质量管理原则：1 以顾客为关注焦点、2 领导作用、3 全员参与、4 过程方法、5 改进、6 循证决策、7 关系管理

质量管理 PDCA-P（戴明环）：plan（策划）->do（执行）->check（检查）->action（改进）->record（固化）

CMMI（软件能力成熟度模型）必考★★★★★

考点：CMMI 五个等级及对应的关键过程域

用途：用于软件过程的改进，用于软件过程的评估，软件能力评鉴。



关键过程域：指明组织改善软件过程能力应关注的区域，并指出为了达到某个成熟度等级所要着手解决的问题。要实现一个关键过程域，就必须达到该关键过程域的所有目标。

能力等级	特点	关键过程区域
初始级	过程不可预测且缺乏控制	
已管理级	过程为项目服务	需求管理、项目计划、配置管理、项目监督与控制、供应商合同管理、度量和分析、过程和产品质量保证
已定义级	过程为组织服务	需求开发、技术解决方案、产品集成、验证、确认组织级过程焦点、组织级过程定义、组织级培训、集成项目管理、风险管理、集成化的团队、决策分析和解决方案、组织级集成环境
定量管理	过程已度量和控制	组织过程性能、定量项目管理
优化级	集中于过程改进和优化	组织级改革与实施、因果分析和解决方案

四、结构化方法

考点：数据流图、数据字典必考（大题），模块内聚和耦合必考，N-S 图和 PAD 图（大题）可能会考。

结构化分析

结构化分析方法是一种面向数据流进行需求分析的方法。模块化、自顶向下、逐层细化，限制使用 goto 语句。

结构化分析方法建立的系统模型是以数据流图为核心，以数据字典和处理逻辑说明为补充。

模块内聚与耦合

高内聚：一个模块内让每个元素都尽可能的紧密相连，充分利用每一个元素的功能，最终实现某个功能。（模块内部）

低耦合：一个完整的系统中，模块与模块之间尽可能的使其独立存在，每个模块尽可能的独立完成某个特定的子功能，模块间的接口尽量少而简单。（模块之间）

1. 模块内聚（以下强度从低到高）★★★★

模块内聚是模块内各处理成分之间的相关关联，作为模块化度量指标之一，指导人们进行模块结构设计的重要概念。

- ① 偶然内聚：一个模块各成分之间毫无关系
- ② 逻辑内聚：几个逻辑上相关的功能被放在同一模块中
- ③ 时间内聚：一个模块完成的功能必须在同一时间内执行（如系统初始化），但这些功能只是因为时间因素关联在一起
- ④ 过程内聚：一个模块内部的处理成分是相关的，而且这些处理必须以特定的次序执行
- ⑤ 通信内聚：一个模块的所有成分都操作同一数据集或生成同一数据集
- ⑥ 顺序内聚：一个模块的各个成分和同一个功能密切相关，而且一个成分的输出作为另一个成分的输入

- ⑦ 功能内聚：模块的所有成分对于**完成单一的功能**都是必须的
2. 模块耦合 ★★★★★
- 耦合是对不同模块间相互依赖程度的度量
- ① **内容耦合**：一个模块直接修改或操作另一个模块的数据
 - ② **数据耦合**：模块间通过**参数传递**基本数据类型的数据
 - ③ 公共耦合：两个以上的模块**共同引用一个全局数据项**
 - ④ 控制耦合：一个模块在界面上传递一个**信号控制另一个模块**，接收信号的模块的动作根据信号值进行调整
 - ⑤ 标记耦合：两个模块至少有一个通过界面传递的公共参数

数据流图

考点：伪码转 N-S 图、PAD 图（大题、示例参考历年真题） ★★★★★★

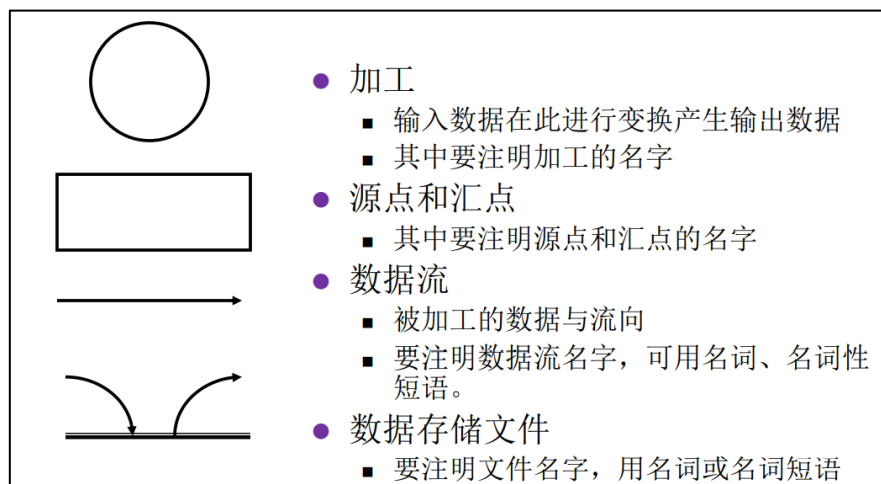
顶层数据流图+数据字典（大题，常和顺序图一块考、示例参考历年真题） ★★★★★★

定义：数据流图是一种描述数据处理的工具。系统接受输入的数据，经过一系列的变换加工，最后输出结果数据。

组成部分：加工、数据流、数据存储、源点和汇点

数据流图画法

1. 图形元素



2. 画数据流图原则：

- ① 一个加工的输出数据流不应与输入数据流同名
 - ② 保持数据守恒。一个加工所有输出数据流中的数据必须能从该加工的输入数据流中直接获得，或者通过该加工能产生的数据
 - ③ 每个加工必须既有输入数据流，又有输出数据流
 - ④ 所有的数据流必须以一个外部实体开始，并以一个外部实体结束
 - ⑤ 外部实体之间不应存在数据流
 - ⑥ 数据流反映“做什么”，不反映“怎么做”，因此数据流名词只能是名词或名词短语
- “由外到内”原则：先确定系统的边界或范围，再考虑系统的内部，先画加工的输入和输出，再画加入的内部。

数据字典（需要注意各个符号对应的含义）

符号	含义	解释
=	被定义为	
+	与	$x=a+b$ ，表示 x 由 a 和 b 组成
[..., ...]	或	$x=[a,b]$ ，表示 x 由 a 或 b 组成
[... ...]	或	$x=[a b]$ ，表示 x 由 a 或 b 组成
{...}	重复	$x=\{a\}$ ，表示 x 由 0 个或多个 a 组成
$m\{...\}n$	重复	$x=3\{a\}8$ ，表示 x 中至少出现 3 次 a ，至多出现 8 次 a
(...)	可选	$x=(a)$ ，表示 a 可在 x 中出现，也可不出现
"..."	基本数据元素	$x="a"$ ，表示 x 可取值为 a 的元素
..	连接符	$x=1..9$ ，表示 x 可取 1 到 9 之中的任一值

例：存折在DD中的定义

- 存折=户名+所号+帐号+开户日+性质+(印密)+1{存取行}50
- 户名=2{字母}24
- 所号="001"..."99999999"
- 开户日=年+月+日
- 性质="1"..."6"
- 印密="0"
- 存取行=日期+(摘要)+支出+存入+余额+操作+复核
- 日期=年+月+日
- 年="00".."99"
- 月="01".."12"
- 日="01".."31"
- 摘要=1{字母}4
- 支出=金额
- 金额="0000000.01"..."9999999.99"
- 操作="00001"..."99999"

五、UML 面向对象建模

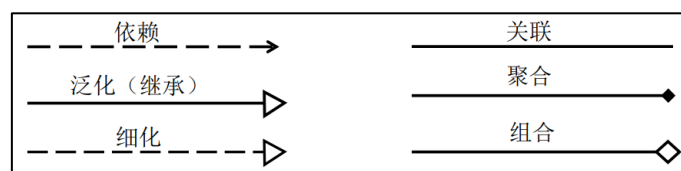
考点：类之间的关系（泛化、聚合），以选择、判断为主；顺序图（建模题）；几乎必考

UML 内容

1. UML 表示法：图形

用例图、类图、对象图、状态图、顺序图、协作图、活动图、构件图、部署图

2. 基本元素：关系 必考★★★★★★

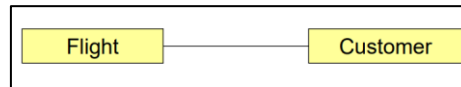


① 依赖：一个元素依赖另一个元素

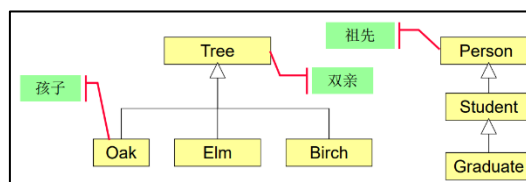
例：实现、使用、实例化、调用、引入、派生、访问...
司机依赖车



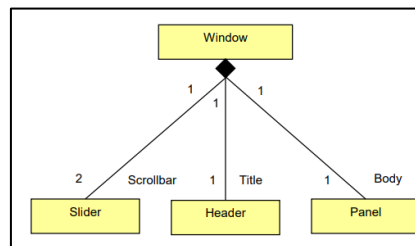
- ② 关联：连接模型元素及链接实例
示例：球员与球队



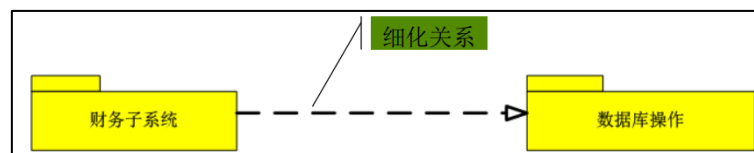
- ③ 泛化：是一种继承关系，表示一般与特殊（一般与特殊）关系，即“一般”元素是“特殊”关系的泛化
例：运输工具与汽车、动物与牛



- ④ 聚合：整体与部分，整体与部分可以分开
例：飞机与发动机、电话机与话筒、学校与老师、公司与法人代表



- ⑤ 细化：同一事物不同级别的两种描述之间的关系



- ⑥ 组合：整体与部分关系，整体与部分不可分开
例：人与人的心脏

3. 基本元素：参与者

参与者：与系统、子系统或类交互的外部人员、进程或事务。



如何确定参与者：

- ⑦ 谁使用系统的主要功能（主参与者）
- ⑧ 谁需要从系统获得对日常工作的支持和服务
- ⑨ 需要谁维护管理系统的日常运行（副参与者）

- ⑩ 系统需要控制哪些硬件设备
- ⑪ 系统需要与其他哪些系统交互
- ⑫ 谁需要使用系统产生的结果（值）

静态建模

用例图 ★★★最近几年常考

创建用例模型工作包括：定义系统、确定执行者和用例、描述用例、定义用例间的关系、确认模型

用例是系统提供的外部可感知的功能单元，定义清晰的系统行为。描述用户的需求。

如何确定用例

- ⑥ 与系统实现有关的主要问题是什么
- ⑦ 系统需要哪些输入/输出？这些输入/输出从何而来？到哪里去？
- ⑧ 参与者需要系统提供哪些功能？
- ⑨ 参与者是否需要系统对系统中的信息进行读取、创建、修改、删除或存储？

案例：

现有一医院病房监护系统，病症监视器安置在每个病房，将病人的病症信号实时传送到中央监视系统进行分析处理。在中心值班室里，值班护士使用中央监视系统对病员的情况进行监控，根据医生的要求随时打印病人的病情报告，定期更新病历，当病症出现异常时，系统会立即自动报警，并实时打印病人的病情报告，立及更新病历。 要求根据现场情景，对医院病房监护系统进行需求分析，建立系统的用例模型。

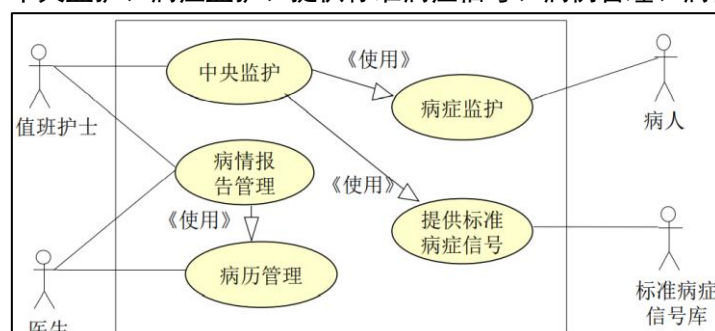
1. 系统功能要求

- ① 监视病员的病症（血压、体温、脉搏等）
- ② 定时更新病历
- ③ 病员出现异常情况时报警
- ④ 随机地产生某一病员的病情报告

2. 角色描述：

角色：病人 角色职责： 提供病症信号 角色职责识别： 负责生成、实时提供各种病症信号。	角色：医生 角色职责： 对病人负责，负责处理病情的变化 角色职责识别： (1)需要系统支持以完成其日常工作 (2)对系统运行结果感兴趣	角色：值班护士 角色职责： 负责监视病人的病情变化 角色职责识别： (1)使用系统主要功能 (2)对系统运行结果感兴趣	角色：标准病症信号库 角色职责： 负责向系统提供病症信号的正常值 角色职责识别： (1)负责保持系统正常运行 (2)与系统交互
---	--	--	--

2. 确定用例：中央监护、病症监护、提供标准病症信号、病例管理、病情报告管理。



用例细化

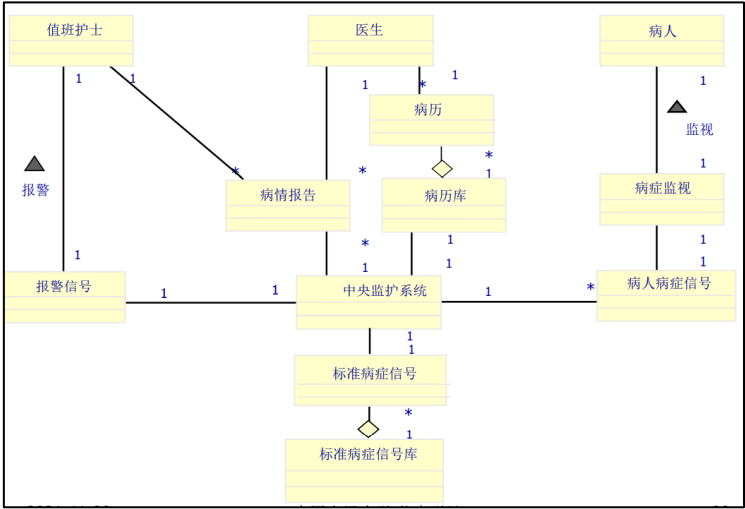
① 中央监护

值班护士	医生	病人	病症监视
用户名 密码	用户名 密码	姓名 性别 年龄 病症	采集频率 病症信号
查看病情报告 () 打印病情报告 ()	查看病情报告 () 要求打印病情报告 () 查看病历 () 要求打印病历 ()	提供病症信号 ()	格式化信号数据 () 采集信号 () 信号组合 ()

中央监护系统	报警信号	标准病症信号库	病历库
输入 输出	声音 灯光 文字	类型 大小 容量	类型 大小 容量
分解信号 () 比较信号 () 报警 () 数据格式化 ()	报警 () 数模转化 ()	提供标准信号 ()	生成病历 () 更新病历 () 查看病历 () 打印病历 ()

病人病症信号	病情报告	病历	标准病症信号
脉搏 血压 体温	标题 格式	格式 病人基本情况 打印时间	脉搏 血压 体温
生成病症信号 ()	生成病情报告 () 查看病情报告 () 打印病情报告 ()	生成病历 () 查看病历 () 打印病历 ()	生成标准信号 ()

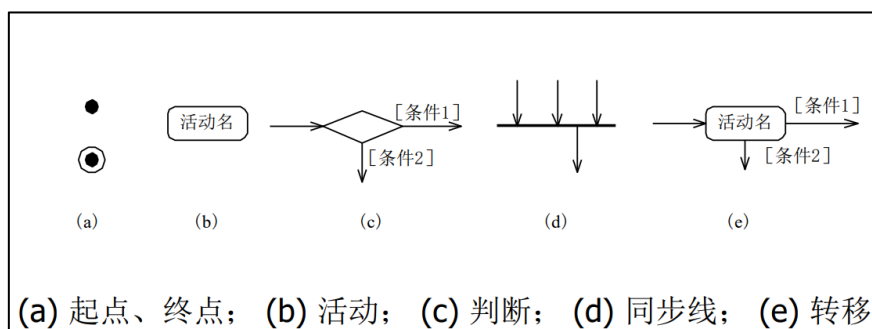
各个类之间的关系



动态建模

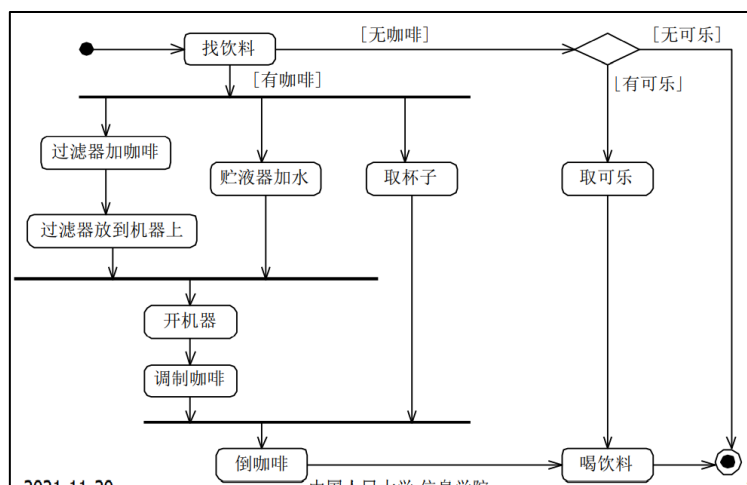
1、活动图

活动图一般不描述所有的运算细节，它显示活动的流，但不显示执行活动的对象。



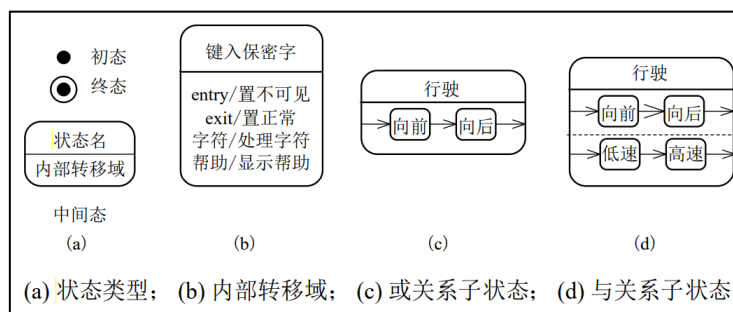
案例：

- [有咖啡] 时， 转移到一个同步线，同时开始 3 个活动：
 - 第一个是“过滤器加咖啡”活动，然后顺序转移到“过滤器放到机器上”活动；
 - 第二个是“贮液器加水”活动；
 - 第三个是“取杯子”活动。
- [无咖啡] 时，则转移到有无可乐的判断。
- [有可乐] 条件时，则执行：
 - “取可乐”活动
 - “喝饮料”活动
- [无可乐] 条件时，则转移到结束。

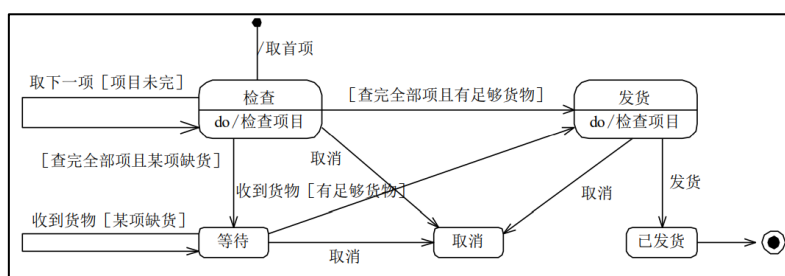


2、状态图

状态图对单个类的对象的生命周期进行建模，描述了对对象时间上的动态行为，每个对象被认为是**事件驱动的孤立实体**。

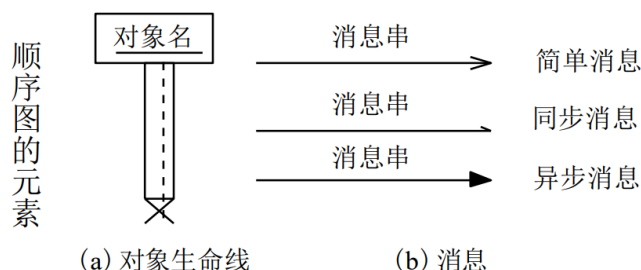


示例：订单类状态图



3、顺序图 ★★★★★（多建模题）

顺序图：描述对象自身及对象间信息传递的视图，重点在消息的时间顺序。



示例：参考历年真题

六、软件测试

考点：白盒测试：几种覆盖、区分强度覆盖，环形复杂度；
黑盒测试：等价类划分、边界值

软件测试活动：需求阶段-制定测试计划、设计阶段-设计测试用例、实现阶段-测试实施、测试阶段-测试工作的展开、验收阶段-验收测试。

单元测试-详细设计；集成测试-概要设计；系统测试-规格说明书；验证测试-需求分析。

测试用例包括要测试的功能、应输入的数据和预期的输出结果。测试用例不仅要检查程序是否做了它应该做的事，还要检查它是否做了不该作的事。

白盒测试

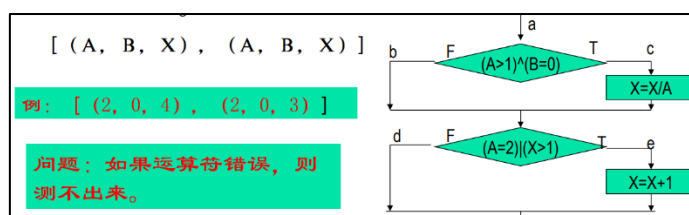
白盒测试是对程序控制结构的测试，依赖对程序细节的了解。允许测试人员利用程序内部的逻辑结构及有关信息，设计或选择测试用例，对程序所有路径进行测试。着重于程序的内部结构及算法。

白盒测试包括：语句覆盖、判定覆盖、条件覆盖、判定-条件覆盖、条件组合覆盖、路径覆盖（由弱到强）

覆盖标准	程序结构举例	测试用例应满足的条件	覆盖标准	程序结构举例	测试用例应满足的条件
语句覆盖		$A*B=T, F$	条件覆盖		$A=T, A=F, B=T, B=F$
判定覆盖		$A*B=T, A*B=F$	判定/条件覆盖		$A*B=T, A*B=F, A=T, A=F, B=T, B=F$
			条件组合覆盖		$A=T, B=T, A=T, B=F, A=F, B=T, A=F, B=F$

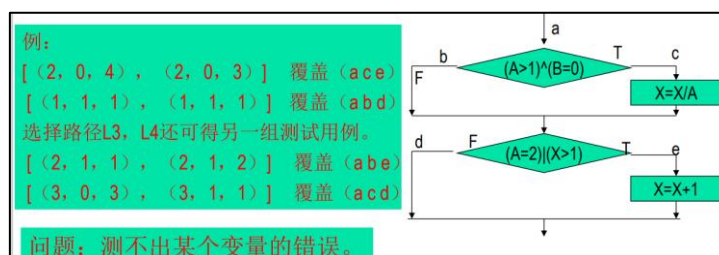
1、语句覆盖

使得每一可执行语句至少执行一次



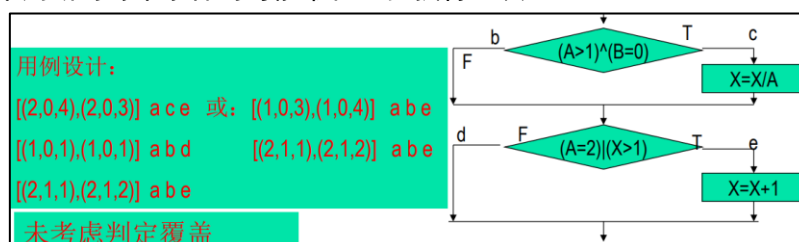
2、判定覆盖

程序中每个判定的取真和取假分支至少经历一次。



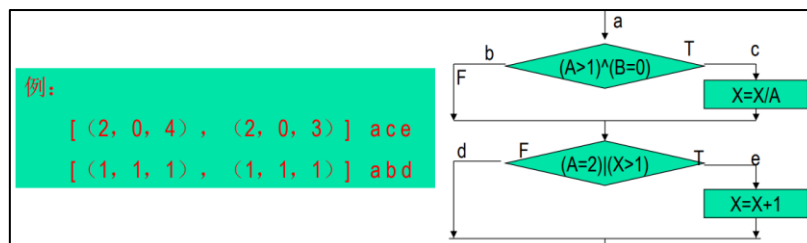
3、条件覆盖

程序中每个判断的每个条件的可能取值至少执行一次



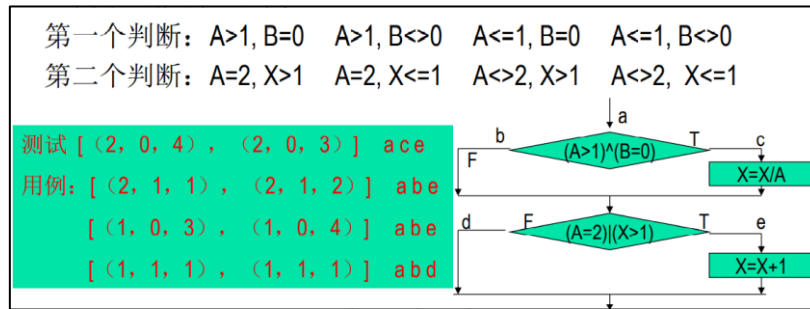
4、判定-条件覆盖

使判断中每个条件的所有可能取值至少执行一次, 同时每个判断的所有结果至少执行一次



5、条件组合覆盖

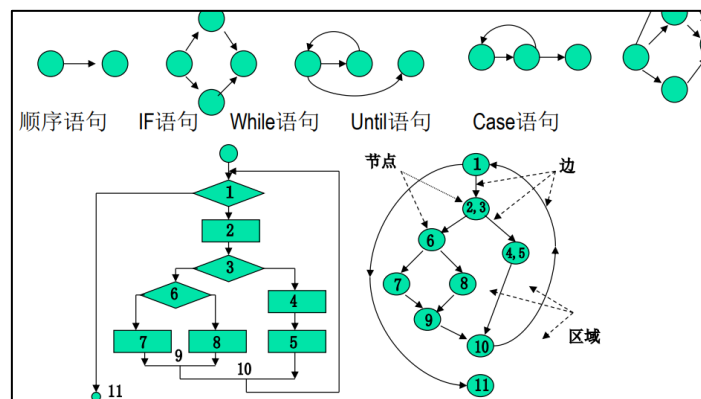
使每个判断的所有可能的条件取值组合至少执行一次



6、路径覆盖测试

基本路径测试用例保证程序的每一个可执行语句至少执行一次。

① 控制流图



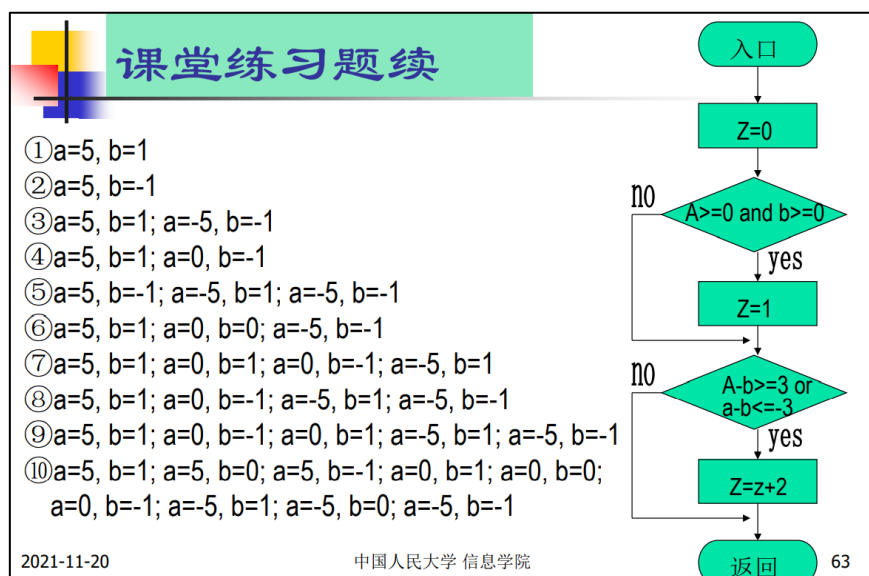
② 环路复杂度

- a) $V(G) = E - N + 2$ E 为边的数量, N 为节点数
 $V(G) = 11 - 9 + 2 = 4$
- b) $V(G) = P + 1$ P 是流图 G 中判定节点的数量
 $V(G) = 3 + 1 = 4$

练习:

根据判定覆盖、条件覆盖、判定 条件覆盖、条件组合覆盖、路径覆盖等 五种覆盖标准, 从供选择的答案中分别找出满足相应覆盖标准的最小的测试数据组。

答案: ④、③、⑥、⑧、⑦



黑盒测试

黑盒测试注重测试软件功能性需求。用于发现功能不对或遗漏、界面错误、数据结构或外部数据库访问错误、性能错误、初始化和终止错误。

七种黑盒测试方法：等价类划分、边界值分析、错误推测法、因果图、判定表、状态迁移法、正交实验法。

等价划分

等价划分原则：有效等价类、无效等价类、按照数值集合划分、按照限制条件或规则划分、细分等价类

示例：某城市电话号码由三部分组成：地区码：空白或 3 位数字；前缀：非 ‘0’ 或 ‘1’ 开头的三位数字；后缀：4 位数字

③ 等价类划分

输入条件	有效等价类	无效等价类
地区码	空白 (1) 3位数字 (2)	有非数字字符 (5) 少于3位数字 (6) 多于3位数字 (7)
前缀	从200到999之间的3位数字 (3)	有非数字字符 (8) 起始位为'0' (9) 起始位为'1' (10) 少于3位数字 (11) 多于3位数字 (12)
后缀	4位数字 (4)	有非数字字符 (13) 少于4位数字 (14) 多于4位数字 (15)

④ 确定测试用例

测试数据	测试范围	期望结果
() 276-2345	等价类 (1) (3) (4)	有效
(635) 805-9321	等价类 (2) (3) (4)	有效

⑤ 确定无效结果测试用例

测试数据	测试范围	期望结果
(20A) 123-4567	无效等价类 (5)	无效
(33) 234-5678	无效等价类 (6)	无效
(7777) 345-6789	无效等价类 (7)	无效
.	.	.
.	.	.
.	. (15)	无效

本例的 15 个等价类至少需要 13 个测试用例

边界值分析

边界值设计测试用例原则：如输入条件代表以 a 和 b 为边界的范围，测试用例用包含 a、b、略大于 a、略大于 b 的值

例：邮件收费规定 1~5 kg 收费 2 元，则应对：0.9, 1, 5, 5.1kg 或 0.99, 1, 5, 5.01kg 设计测试用例。

等价类划分、边界值都着重考虑输入条件，但没有考虑输入条件的各种组合、输入条件之间的相互制约关系，因果图法充分考虑输入情况的各种组合及输入条件之间的相互制约关系。

七、项目管理

估算

基于经验、基于模型

规模估算：代码行、功能点

度量

定义：根据一定的规则，将数字或符号赋予系统、构件、过程等实体的特定属性，从而能清晰地理解该实体及其属性。简言之，度量就是对事物属性的量化。

目的：理解、预测、评估、改进

重要性：度量活动可以对软件开发项目状态和产品质量给予量化的表示，为加强和改进研发工作提供详细的指导

如何进行度量活动：

① 识别目标 ② 定义度量过程 ③ 数据收集 ④ 数据分析与反馈 ⑤ 过程改进

风险

定义：在一定条件下和一定时间内可能发生各种结果的变化程度。

本质：不确定性（不存在概率 100% 的风险）、危害性（风险变成现实，会产生恶性后果或损失）

风险预测：风险发生的可能性或概率；风险发生造成的后果

处理风险策略：① 风险避免 ② 风险监控 ③ 风险管理及意外事件计划

八、软件质量

考点：软件质量保证

软件质量

定义：与软件产品满足明确或隐含需求的能力有关的特征和特性的总和。

软件质量特性：功能性、可靠性、易使用性、效率、可维护性、可移植性

特 性	子 特 性	特 性	子 特 性
功能性 Functionality	适合性 Suitability	效率 Efficiency	时间特性 Time behaviour
	准确性 Accuracy		资源特性 Resource behaviour
	互操作性 Interoperability	可维护性 Maintainability	易分析性 Analysability
	依从性 Compliance		易更改性 Changeability
	安全性 Security		稳定性 Stability
可靠性 Reliability	成熟性 Maturity	可移植性 Portability	易测试性 Testability
	容错性 Fault tolerance		适应性 Adaptability
	易恢复性 Recoverability		易安装性 Installability
易使用性 Usability	易理解性 Understandability		遵循性 Conformance
	易学习性 Learnability		易替换性 Replaceability
	易操作性 Operability		

可靠性：一个程序按照用户的要求和设计的目标，执行其功能的正确程度；正确性、健壮性，系统在正常情况下不正常工作，异常情况下适当处理，不会造成严重的损失。

可维护性：软件能够被理解、校正、适应及增强功能的容易程度。

可移植性：软件在不同环境下的运行能力，为提高可移植性，应使软件尽量不与具体的环境关联，即提高软件的设备独立性。

软件质量管理活动

质量保证：建立起组织的质量体系，这是生产高质量软件的保证

质量规划：为特定的项目编制质量计划

质量控制：定义并设计质量过程，确保软件开发团队严格遵守项目质量规程和标准

软件质量保证

定义：质量保证是为项目生存周期内的软件过程和软件产品提供适当保障的过程，目的是使它们符合所规定的需求，并遵循已建立计划。

四个活动：过程实现，产品保证，过程保证，质量体系保证。

软件质量两个阶段：①设计质量 ② 符合质量

技术评审

软件质量保证的活动之一是技术评审。技术评审是一种静态分析，评审对象通常是技术文档、计划、测试用例、测试数据、测试结果等，是质量控制最有效的手段之一。常见的技术评审包括走查、轮查、正式的同行评审等。

目的：尽早地发现工作成果中的缺陷，并帮助开发人员及时消除缺陷，从而有效地提高产品的质量，并向管理提供证据，以表明产品是否满足规范说明并遵从标准，而且可以控制变更。